# UNIT I INTRODUCTION TO DATA STRUCTURES

***Abstract Data Types (ADTs) – List ADT – array-based implementation – linked list implementation —singly linked lists- doubly-linked lists –circularly linked lists- applications of lists –Polynomial Manipulation.***

**Data structure** A data structure is a specialized format for organizing and storing data. General data structure types include the array, the file, the record, the table, the tree, and so on. Any data structure is designed to organize data to suit a specific purpose so that it can be accessed and worked with in appropriate ways

**Characteristics of data structures:**

1. It depicts the logical representation of data in computer memory.
2. It represents the logical relationship between the various data elements.
3. It helps in the efficient manipulation of stored data elements.
4. It allows the programs to process the data in an efficient manner.

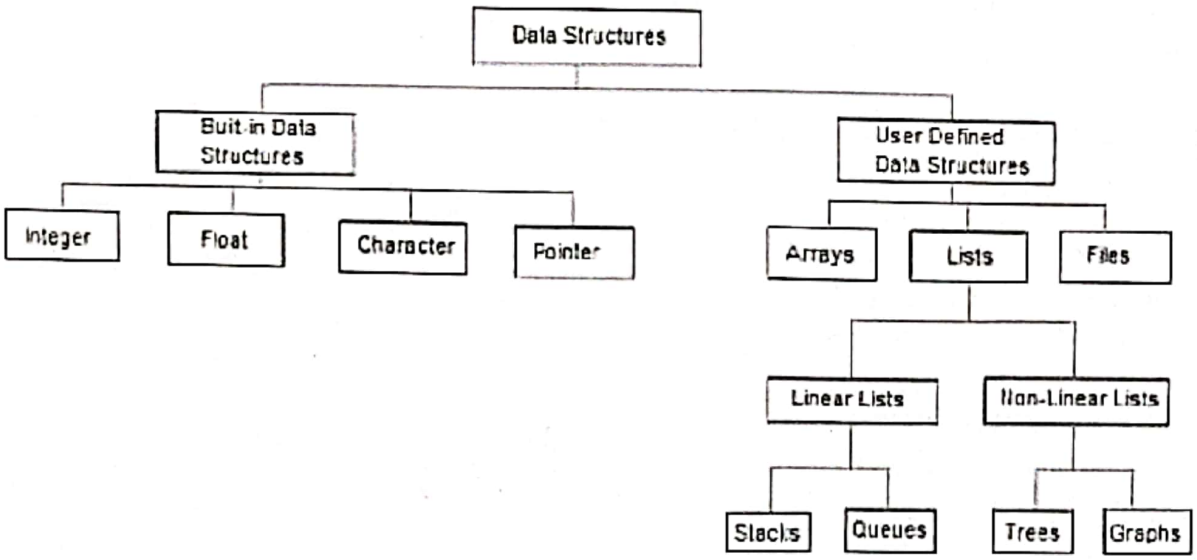
**Operations on Data Structures:**

1. Traversal
2. Search
3. Insertion
4. Deletion

## 1. ABSTRACT DATA TYPE

In computer science, an abstract data type (ADT) is a mathematical model for data types where a data type is defined by its behavior (semantics) from the point of view of a user of the data, specifically in terms of possible values, possible operations on data of this type, and the behavior of these operations. When a class is used as a type, it is an abstract type that refers to a hidden representation. In this model an ADT is typically implemented as a class, and each instance of the ADT is usually a n object of that class.

In ADT all the implementation details are hidden




Scanned with OKEN Scanner

•**Linear data structures** are the data structures in which data is arranged in a list or in a sequence.

•**Non-linear data structures** are the data structures in which data may be arranged in a hierarchical manner

## 2. LIST ADT

List is an ordered set of elements.

The general form of the list is A1, A2, ……,

AN A1 – 1st element of the list

A2- 2nd element of the

list N –Size of the list

If the element at position i is Ai, then its successor is Ai+1 and its predecessor is Ai-1

**Various operations performed on List**

**Insert (X, 5)-** Insert the element X after the position 5.

**Delete (X)** - The element X is deleted

**Find (X)** - Returns the position of X.

**Next (i)** - Returns the position of its successor element i+1.

**Previous (i)-** Returns the position of its predecessor i-1.

**Print list** - Contents of the list is displayed.

**Makeempty**- Makes the list empty.

**Implementation of list ADT:**

1. Array based Implementation
2. Linked List based implementation

## 3. ARRAY IMPLEMENTATION OF LIST

**Array** is a collection of specific number of same type of data stored in consecutive memory locations. Array is a static data structure i.e., the memory should be allocated in advance and the size is fixed. This will waste the memory space when used space is less than the allocated space.

Insertion and Deletion operation are expensive as it requires more data movements Find and Print list operations takes constant time.

**The basic operations performed on a list of elements are**

1. Creation of List.
2. Insertion of data in the List
3. Deletion of data from the List
4. Display all data in the List
5. Searching for a data in the list

**Declaration of Array:**

```
#define maxsize 10
int list[maxsize], n ;
```



Scanned with OKEN Scanner

### 3.1. CREATE OPERATION:

Create operation is used to create the list with "n" number of elements. If "n" exceeds the array's maxsize, then elements cannot be inserted into the list. Otherwise, the array elements are stored in the consecutive array locations (i.e.) list [0], list [1] and so on.

**Pseudocode:**

```
void Create ( )
{
        int i;
        printf("\nEnter the number of elements to be added in the list:\t");
        scanf("%d",&n);
        printf("\nEnter the
        array elements:\t");
        for(i=0;i<n;i++)
                scanf("%d",&list[i]);
}
```

If n=6, the output of creation is as follows: list[6]

| 20 | 10 | 30 | 40 | 50 | 60 |
|---|---|---|---|---|---|
| A [0] | A [1] | A [2] | A [3] | A [4] | A [5] |

### 3.2. INSERT OPERATION

Insert operation is used to insert an element at particular position in the existing list. Inserting the element in the last position of an array is easy. But inserting the element at a particular position in an array is quite difficult since it involves all the subsequent elements to be shifted one position to the right.

**Routine to insert an element in the array:**

```
void Insert( )
{
      int i,data,pos;
      printf("\nEnter the data to be inserted:\t");
      scanf("%d",&data);
      printf("\nEnter the position at which element to be inserted:\t");
      scanf("%d",&pos);
      if (pos==n)
      printf ("Array overflow");
       for (i = n-1; i >= pos-1 ; i--)
      {
      list[i+1] = list[i];
      }
       list[pos-1] = data; n=n+1;
       Display();
       }
```

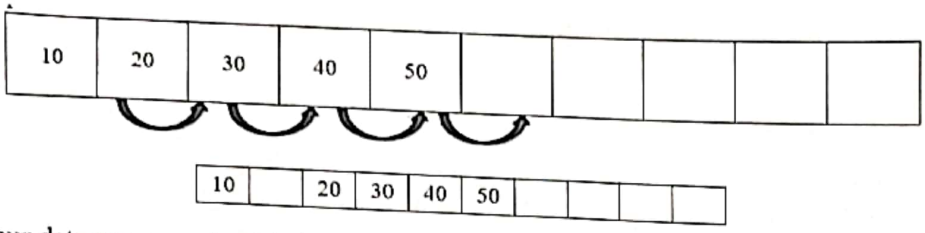
Consider an array with 5 elements [ max elements = 10]

| 10 | 20 | 30 | 40 | 50 | | | | | |
|---|---|---|---|---|---|---|---|---|---|



Scanned with OKEN Scanner

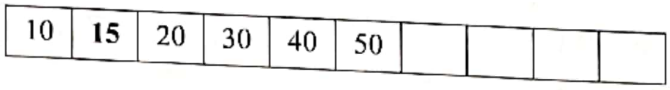
If data 15 is to be inserted in the 2nd position than 50 has to be moved to next index position, 40 has to be moved to 50 position, 30 has to be moved to 40 position and 20 has to be moved to 30 position.







After this four data movement, 15 is inserted in the 2nd position of the array.




### 3.3. DELETION OPERATION

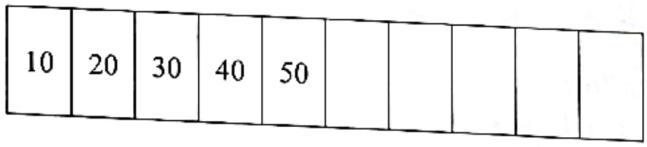
Deletion is the process of removing an element from the array at any position.

Deleting an element from the end is easy. If an element is to be deleted from any particular position, it requires all subsequent element from that position is shifted one position towards left.

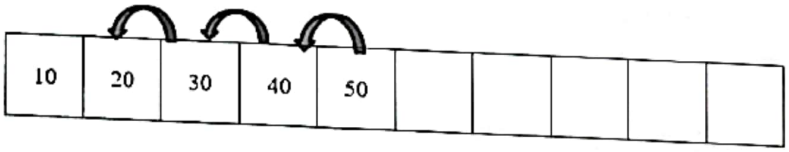
**Routine to delete an element in the array:**

```
void Deletc( )
{
   int i, pos ;
   printf("\nEnter the position of the data to be deleted:\t");
   scanf("%d",&pos);
   printf("\nThe data deleted is:\t %d", list[pos-1]);
   for(i=pos-1;i<n-1;i++)
   {
         list[i]=list[i+1];
   }
   n=n-1;
   Display();
}
```

Consider an array with 5 elements [ max elements = 10 ]




If data 20 is to be deleted from the array, then 30 has to be moved to data 20 position, 40 has to bemoved to data 30 position and 50 has to be moved to data 40 position




After these 3 data movements, data 20 is deleted from the 2nd position of the array.



Scanned with OKEN Scanner

| 10 | 30 | 40 | 50 | | | | | | |
|---|---|---|---|---|---|---|---|---|---|

### 3.4. DISPLAY OPERATION/TRAVERSING A LIST

Traversal is the process of visiting the elements in a array.

Display( ) operation is used to display all the elements stored in the list. The elements are stored from the index 0 to n - 1. Using a for loop, the elements in the list are viewed

**Routine to traverse/display elements of the array:**

```
void display( )
{
        int i;
        printf("\n**********Elements in the array**********\n");
        for(i=0;i<n;i++)
                printf("%d\t",list[i]);
}
```

### 3.5. SEARCH OPERATION

Search( ) operation is used to determine whether a particular element is present in the list or not. Input the search element to be checked in the list.

**Routine to search an element in the array:**

```
void Search( )
{
        int search,i,count = 0;
        printf("\nEnter the element to be searched:\t");
        scanf("%d",&search);
        for(i=0;i<n;i++)
        {
        if(search == list[i]);
                count++;
        }
        if(count==0)
                printf("\nElement not present in the list"); else
        printf("\nElement present in the list");
}
```

**Advantages of array implementation:**

1. The elements are faster to access using random access
2. Searching an element is easier

**Limitation of array implementation**

1. An array store its nodes in consecutive memory locations.
2. The number of elements in the array is fixed and it is not possible to change the number of elements.
3. Insertion and deletion operation in array are expensive. Since insertion is performed by pushing the entire array one position down and deletion is performed by shifting the entire array one position up.

**Applications of arrays:**

Arrays are particularly used in programs that require storing large collection of similar type data elements.



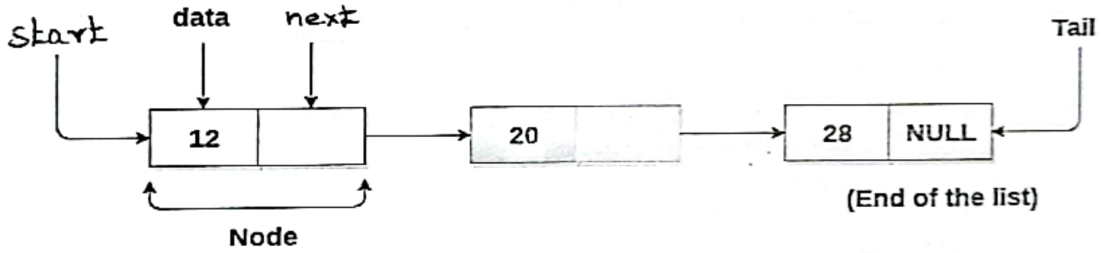
Scanned with OKEN Scanner

**Differences between Array based and linked based implementation**

| | Array | Linked List |
|---|---|---|
| Definition | Array is a collection of elements having same data type with common name | Linked list is an ordered collection of elements which are connectedby links/pointers |
| Access | Elements can be accessed using index/subscript, random access | Sequential access |
| Memory structure | Elements are stored in contiguous memory locations | Elements are stored at available memory space |
| Insertion & Deletion | Insertion and deletion take more time in array | Insertion and deletion are fast and easy |
| Memory Allocation | Memory is allocated at compile time i.e static memory allocation | Memory is allocated at run time i.e dynamic memory allocation |
| Types | 1D,2D, multi-dimensional | SLL, DLL circular linked list |
| Dependency | Each element is independent | Each node is dependent on each other as address part contains address of next node in the list |

## 4. SINGLY LINKED LIST

- Linked List can be defined as collection of objects called nodes that are randomly stored in the memory.
- A node contains two fields i.e. data stored at that particular address and the pointer which contains the address of the next node in the memory.
- The last node of the list contains pointer to the null.




### Uses of Linked List

- The list is not required to be contiguously present in the memory. The node can reside anywhere in the memory and linked together to make a list. This achieves optimized utilization of space.
- list size is limited to the memory size and doesn't need to be declared in advance.
- Empty node cannot be present in the linked list.
- We can store values of primitive types or objects in the singly linked list.

### Structure of Node

```
struct node
{
   int data;
   struct node *next;
};
struct node *start, *ptr;
```



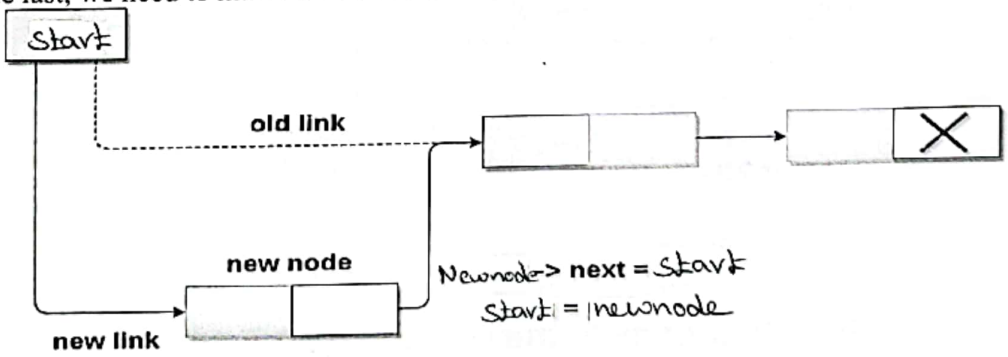
Scanned with OKEN Scanner

## OPERATIONS ON SINGLY LINKED LIST:

| S. No | Operation | Description |
|---|---|---|
| 1 | Insertion at beginning | It involves inserting any element at the front of the list. We just need to a few link adjustments to make the new node as the head of the list. |
| 2 | Insertion at end of the list | It involves insertion at the last of the linked list. The new node can be inserted as the only node in the list or it can be inserted as the last one. Different logics are implemented in each scenario. |
| 3 | Insertion after specified node | It involves insertion after the specified node of the linked list. We need to skip the desired number of nodes in order to reach the node after which the new node will be inserted. |
| 4 | Deletion at beginning | It involves deletion of a node from the beginning of the list. This is the simplest operation among all. It just need a few adjustments in the node pointers. |
| 5 | Deletion at the end of the list | It involves deleting the last node of the list. The list can either be empty or full. Different logic is implemented for the different scenarios. |
| 6 | Deletion after specified node | It involves deleting the node after the specified node in the list. we need to skip the desired number of nodes to reach the node after which the node will be deleted. This requires traversing through the list. |
| 7 | Traversing | In traversing, we simply visit each node of the list at least once in order to perform some specific operation on it, for example, printing data part of each node present in the list. |
| 8 | Searching | In searching, we match each element of the list with the given element. If the element is found on any of the location then location of that element is returned otherwise null is returned. . |

### 4.1 INSERTION IN SINGLY LINKED LIST AT BEGINNING

- Allocate the space for the new node and store data into the data part of the node.
  **Newnode = (struct node *) malloc(sizeof(struct node *));**
- Make the link part of the new node pointing to the existing first node of the list. This will be done by using the following statement.
- At the last, we need to make the new node as the first node of the list.






Scanned with OKEN Scanner

**Pseudocode:**

```
void insert_begin()
{
    struct node *Newnode;
    Newnode=(struct node *)malloc(sizeof(struct node));
    printf("\nEnter the data value for the node:\t" );
    scanf("%d",&Newnode->data);
    Newnode->next =NULL;
    if(start==NULL)
    {
        start=Newnode;
    }
    else
    {
        Newnode->next=start;
        start=Newnode;
    }
}
```

### 4.2 INSERTION IN SINGLY LINKED LIST AT THE END

In order to insert a node at the last, there are two following scenarios which need to be mentioned.

1. The node is being added to an empty list
2. The node is being added to the end of the linked list.

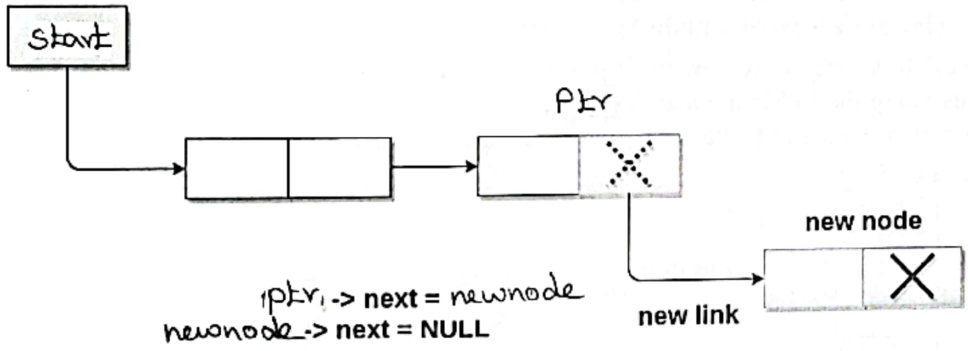
**CASE 1: The node is being added to an empty list.**

The condition (start == NULL) gets satisfied. Hence, we just need to allocate the space for the node by using malloc statement in C. Data and the link part of the node are set. Since, Newnode is the only node that will be inserted in the list hence, we need to make this node pointed by the start pointer of the list.

**CASE 2: The node is being added to the end of the linked list.**

The condition (start = NULL) would fail, since start is not null. Now, we need to declare a temporary pointer ptr in order to traverse through the list. ptr is made to point the first node of the list. Then, traverse through the entire linked list.

At the end of the loop, the ptr will be pointing to the last node of the list. Now, allocate the space for the new node, and assign the item to its data part. Since, the new node is going to be the last node of the list hence, the next part of this node needs to be pointing to the null. We need to make the next part of the ptr node (which is currently the last node of the list) point to the new node.




**Inserting node at the last into a non-empty list**



Scanned with OKEN Scanner

**Pseudocode:**

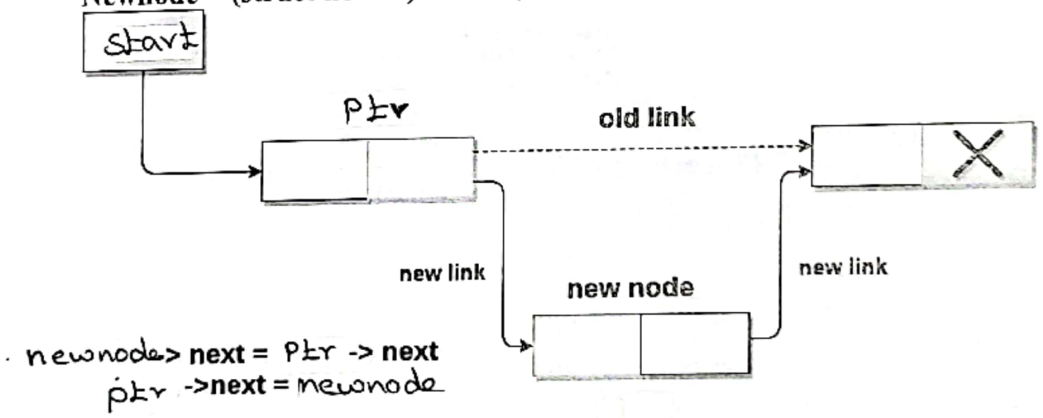
```
void insert_end()
{
    struct node *Newnode,*ptr;
    Newnode=(struct node *)malloc(sizeof(struct node));
    printf("\nEnter the data value for the node:\t" );
    scanf("%d",&Newnode->data );
    Newnode->next =NULL;
    if(start==NULL)
    {
        start=Newnode;
    }
    else
    {
        ptr=start;
        while(ptr->next !=NULL)
        {
            ptr=ptr->next ;
        }
        ptr->next =Newnode;
    }
}
```

## 4.3 INSERTION IN SINGLY LINKED LIST AT SPECIFIED POSITION

In order to insert an element after the specified number of nodes into the linked list, we need to skip the desired number of elements in the list to move the pointer at the position at which the node will be inserted.

- Allocate the space for the new node and store data into the data part of the node.

**Newnode = (struct node *) malloc(sizeof(struct node *));**




**Pseudocode:**

```
void insert_pos()
{
    struct node *ptr,*Newnode;
    int i,pos;
    Newnode=(struct node *)malloc(sizeof(struct node));
    printf("\nEnter the position for the new node to be inserted:\t");
    scanf("%d",&pos);
    printf("\nEnter the data value of the node:\t");
    scanf("%d",&Newnode->data) ;
```

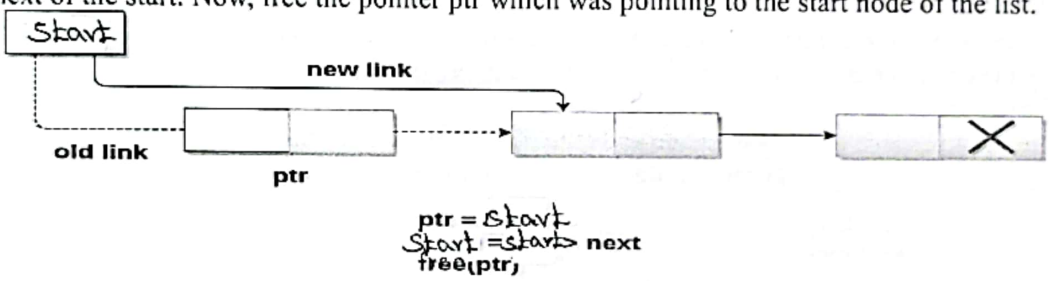


Scanned with OKEN Scanner

```
    Newnode->next=NULL;
    if(pos==0)
    {
        Newnode->next=start;
        start=Newnode;
    }
    else
    {
        for(i=0,ptr=start;i<pos-1;i++)
        {
          ptr=ptr->next;
            if(ptr==NULL)
            {
                printf("\nPosition not found \n");
                return;
            }
        }
        Newnode->next =ptr->next ;
        ptr->next=Newnode;
    }
}
```

### 4.4 DELETION IN SINGLY LINKED LIST AT THE BEGINNING

Since the first node of the list is to be deleted, therefore, we just need to make the start, point to the next of the start. Now, free the pointer ptr which was pointing to the start node of the list.




**Deleting a node from the beginning**

**Pseudocode:**

```
 void delete_begin()
 {
      struct node *ptr;
      if(start==NULL)
      {
          printf("\nList is Empty:\n");
          return;
      }
      else
      {
          ptr=start;
          start=start->next ;
          printf("\nThe deleted element is :%d\t",ptr->data);
          free(ptr);
      }
 }
```



Scanned with OKEN Scanner

## 4.5 DELETION IN SINGLY LINKED LIST AT THE END

There are two scenarios in which, a node is deleted from the end of the linked list.

**CASE 1: There is only one node in the list and that needs to be deleted.**

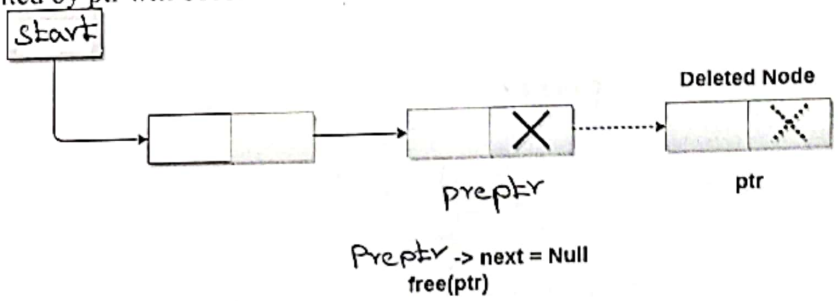
The condition start → next = NULL will survive and therefore, the only node start of the list will be assigned to null.

**CASE 2: There are more than one node in the list and the last node of the list will be deleted.**

The condition start → next = NULL would fail and therefore, we have to traverse the node in order to reach the last node of the list.

For this purpose, just declare a temporary pointer ptr and assign it to start of the list. We also need to keep track of the second last node of the list. For this purpose, two pointers ptr and preptr will be used where ptr will point to the last node and preptr will point to the second last node of the list.

Now, we just need to make the pointer preptr→next point to the NULL and the last node of the list that is pointed by ptr will become free.




**Deleting a node from the last**

**Pseudocode:**

```
void delete_end()
{
    struct node *preptr,*ptr;
    if(start==NULL)
    {
        printf("\nList is Empty:");
        exit(0);
    }
    else if(start->next ==NULL)
    {
        ptr=start;
        //start=NULL;
        printf("\nThe deleted element is:%d\t",ptr->data);
        free(ptr);
    }
    else
    {
        ptr=start;
        while(ptr->next!=NULL)
        {
            preptr=ptr;
            ptr=ptr->next;
        }
        preptr->next=NULL;
```



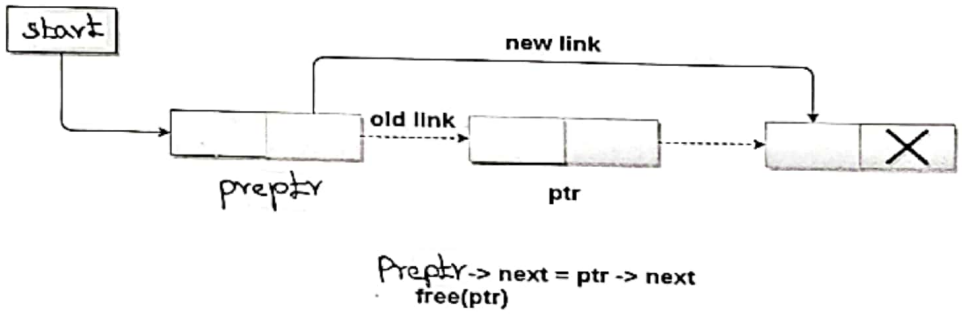
Scanned with OKEN Scanner

```
                printf("\nThe deleted element is:%d\t",ptr->data);
                free(ptr);
            }
        }
```

## 4.6 DELETION IN SINGLY LINKED LIST AT THE SPECIFIED NODE

In order to delete the node, which is present at the specified node, we need to skip the desired number of nodes to reach the node after which the node will be deleted. We need to keep track of the two nodes. The one which is to be deleted the other one if the node which is present before that node. For this purpose, two pointers are used: ptr and preptr.

Now, our task is almost done, we just need to make a few pointer adjustments. Make the next of preptr (points to the specified node) point to the next of ptr (the node which is to be deleted).




**Deletion a node from specified position**

**Pseudocode:**

```
void delete_pos()
{
    int i,pos;
    struct node *preptr,*ptr;
    if(start==NULL)
    {
        printf("\nThe List is Empty:\n");
        exit(0);
    }
    else
    {
        printf("\nEnter the position of the node to be deleted:\t");
        scanf("%d",&pos);
        if(pos==0)
        {
            ptr=start;
            start=start->next ;
            printf("\nThe deleted element is:%d\t",ptr->data  );
            free(ptr);
        }
        else
        {
            ptr=start;
            for(i=0;i<pos;i++)
            {
              preptr=ptr;
              ptr=ptr->next ;
```



Scanned with OKEN Scanner

```
                    if(ptr==NULL)
                    {
                        printf("\nPosition not Found:\n");
                        return;
                    }
                }
                preptr->next =ptr->next ;
                printf("\nThe deleted element is:%d\t",ptr->data );
                free(ptr);
            }
        }
    }
```

## 4.7 TRAVERSING IN SINGLY LINKED LIST

Traversing is the most common operation that is performed in almost every scenario of singly linked list. Traversing means visiting each node of the list once in order to perform some operation.

**Pseudocode:**

```
void display()
{
    struct node *ptr;
    if(start==NULL)
    {
        printf("\nList is empty:\n");
        return;
    }
    else
    {
        ptr=start;
        printf("\nThe List elements are:\n");
        while(ptr!=NULL)
        {
            printf("%d\t",ptr->data );
            ptr=ptr->next ;
        }
    }
}
```

## 4.8 SEARCHING IN SINGLY LINKED LIST

Searching is performed in order to find the location of a particular element in the list. Searching any element in the list needs traversing through the list and make the comparison of every element of the list with the specified element. If the element is matched with any of the list element then the location of the element is returned from the function.

**Pseudocode:**

```
void search()
{
    struct node *ptr;
    int item,i=0,flag;
    ptr = start;
    if(ptr == NULL)
    {
        printf("\nEmpty List\n");
    }
```

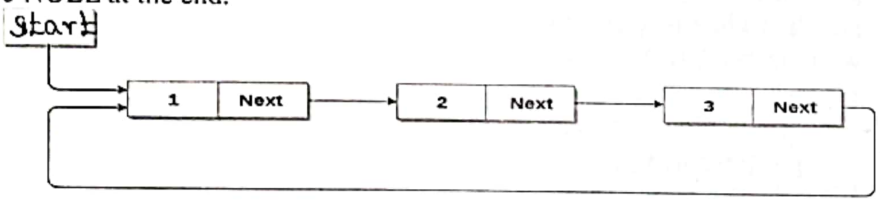


Scanned with OKEN Scanner

```
        else
        {
            printf("\nEnter item which you want to search?\n");
            scanf("%d",&item);
            while (ptr!=NULL)
            {
                if(ptr->data == item)
                {
                    printf("item found at location %d ",i+1);
                    flag=0;
                }
                else
                {
                    flag=1;
                }
                i++;
                ptr = ptr -> next;
            }
            if(flag==1)
            {
                printf("Item not found\n");
            }
        }
    }
```

## 5. CIRCULAR SINGLY LINKED LIST

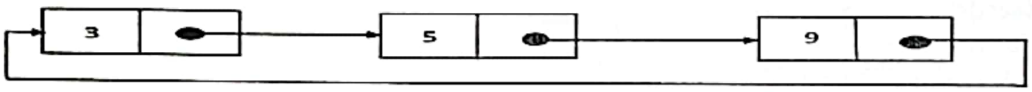
The **circular linked list** is a linked list where all nodes are connected to form a circle. In a circular linked list, the first node and the last node are connected to each other which forms a circle. There is no NULL at the end.




**Circular Singly Linked List**

**There are generally two types of circular linked lists:**

- **Circular singly linked list:** In a circular Singly linked list, the last node of the list contains a pointer to the first node of the list. We traverse the circular singly linked list until we reach the same node where we started. The circular singly linked list has no beginning or end. No null value is present in the next part of any of the nodes.

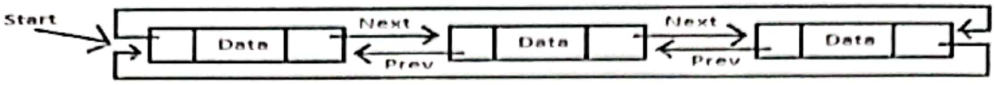



**Representation of Circular singly linked list**

- **Circular Doubly linked list:** Circular Doubly Linked List has properties of both doubly linked list and circular linked list in which two consecutive elements are linked or connected by the previous and next pointer and the last node points to the first node by the next pointer and also the first node points to the last node by the previous pointer.

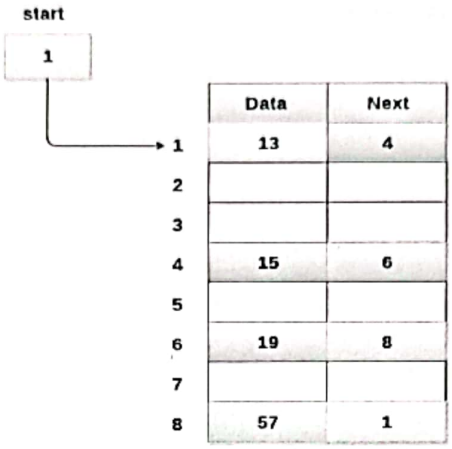


Scanned with OKEN Scanner




**Representation Of Circular Doubly Linked List**

## MEMORY REPRESENTATION OF CIRCULAR LINKED LIST:




**Memory Representation of a circular linked list**

## OPERATIONS ON CIRCULAR SINGLY LINKED LIST:

| S. No | Operation | Description |
|---|---|---|
| 1 | Insertion at beginning | Adding a node into circular singly linked list at the beginning. |
| 2 | Insertion at the end | Adding a node into circular singly linked list at the end. |
| 3 | Deletion at beginning | Removing the node from circular singly linked list at the beginning. |
| 4 | Deletion at the end | Removing the node from circular singly linked list at the end. |
| 5 | Searching | Compare each node element with the given item and return the location at which the item is present in the list otherwise return null. |
| 6 | Traversing | Visiting each element of the list at least once in order to perform some specific operation. |

### 5.1 INSERTION INTO CIRCULAR SINGLY LINKED LIST AT BEGINNING

There are two scenarios in which a node can be inserted in a circular singly linked list at the beginning. Either the node will be inserted in an empty list or the node is to be inserted in an already filled list. Firstly, allocate the memory space for the new node by using the malloc method of C language.

**newnode = (struct node *)malloc(sizeof(struct node));**

**CASE 1:** In the first scenario, the condition **start == NULL** will be true. Since, the list in which, we are inserting the node is a circular singly linked list, therefore the only node of the list will point to itself only. We also need to make the start pointer point to this node.

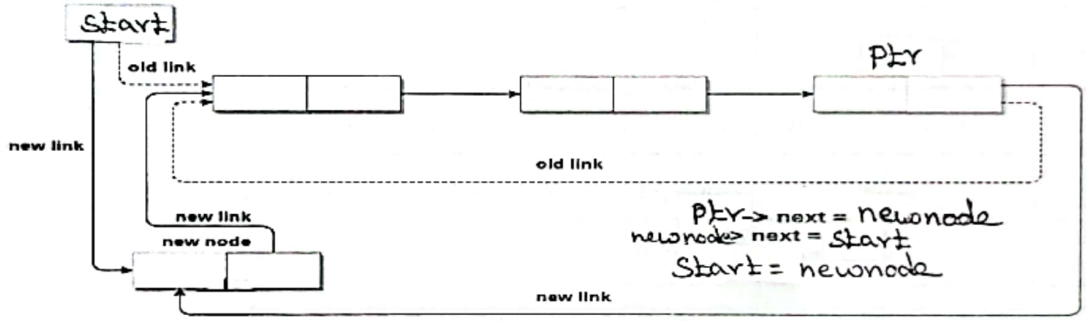


Scanned with OKEN Scanner

**CASE 2:** In the second scenario, the condition start == NULL will become false which means that the list contains at least one node. In this case, we need to traverse the list in order to reach the last node of the list.

**Pseudocode:**

```
void beg_insert()
{
   struct node *newnode,*ptr;
   int val;
   newnode = (struct node *)malloc(sizeof(struct node));
   printf("\nEnter the node data?\t");
   scanf("%d",&val);
   newnode -> data = val;
   if(start == NULL)
   {
      start = newnode;
      newnode -> next = start;
   }
   else
   {
      ptr = start;
      while(ptr->next != start)
         ptr = ptr->next;
      newnode->next = start;
      ptr -> next = newnode;
      start = newnode;
   }
}
```




**Insertion into circular singly linked list at beginning**

## 5.2 INSERTION INTO CIRCULAR SINGLY LINKED LIST AT THE END

Firstly, allocate the memory space for the new node by using the malloc method of C language.

**newnode = (struct node *)malloc(sizeof(struct node));**

**CASE 1:** In the first scenario, the condition **start == NULL** will be true. Since, the list in which, we are inserting the node is a circular singly linked list, therefore the only node of the list will point to itself only. We also need to make the start pointer point to this node.

**CASE 2:** In the second scenario, the condition **start == NULL** will become false which means that the list contains at least one node. In this case, we need to traverse the list in order to reach the last node of the list. At the end of the loop, the pointer **ptr** would point to the last node of the list. Since the new node which is being inserted into the list will be the new last node of the list. Therefore, the existing last node i.e. **ptr** must point to the new node. The new last node of the list will point to the start node of the list.

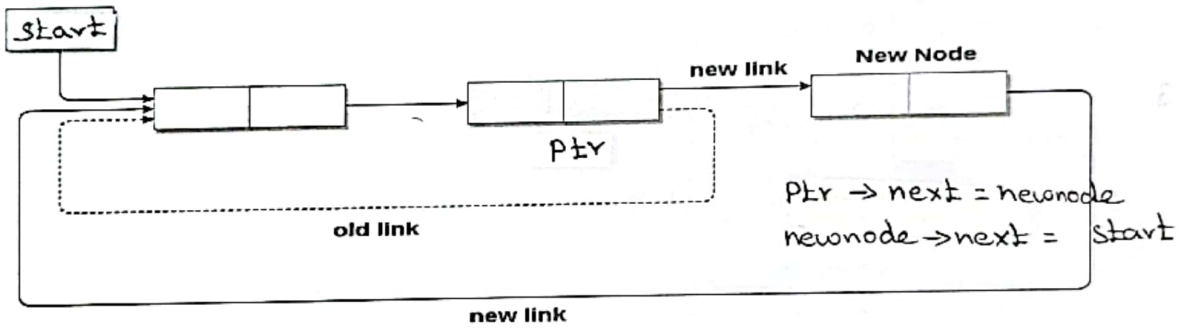
**Pseudocode:**

```
void last_insert()
```



Scanned with OKEN Scanner

```
{
   struct node *newnode,*ptr;
   int val;
   newnode = (struct node *)malloc(sizeof(struct node));
   printf("\nEnter the node data?\t");
   scanf("%d",&val);
   newnode->data = val;
   if(start == NULL)
   {
      start = newnode;
      newnode -> next = start;
   }
   else
   {
      ptr = start;
      while(ptr -> next != start)
      {
         ptr = ptr -> next;
      }
      ptr -> next = newnode;
      newnode -> next = start;
   }
}
```




**Insertion into circular singly linked list at end**

### 5.3 DELETION IN CIRCULAR SINGLY LINKED LIST AT BEGINNING

In order to delete a node in circular singly linked list, we need to make a few pointer adjustments. There are three scenarios of deleting a node from circular singly linked list at beginning.

**CASE 1: The list is Empty**

If the list is empty then the condition **start == NULL** will become true, in this case, we just need to print **underflow** on the screen and make an exit.

**CASE 2: The list contains a single node**

If the list contains a single node, then, the condition **start → next == head** will become true. In this case, we need to delete the entire list and make the start pointer free.

**CASE 3: The list contains more than one node**

If the list contains more than one node then, in that case, we need to traverse the list by using the pointer **ptr** to reach the last node of the list. At the end of the loop, the pointer ptr points to the last node of the list. Since the last node of the list points to the start node of the list. Therefore, this will be changed as now, the last node of the list will point to the next of the start node. Now, free the head pointer by using the free() method in C language. Make the node pointed by the next of the last node, the new head



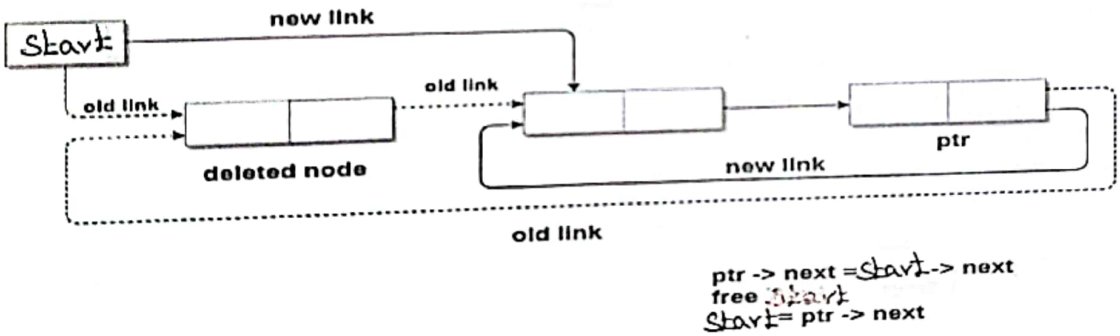
Scanned with OKEN Scanner

of the list.

**Pseudocode:**

```
void begin_delete()
{
   struct node *ptr;
   if(start == NULL)
   {
      printf("\nUNDERFLOW");
   }
   else if(start->next == start)
   {
      start = NULL;
      free(start);
      printf("\nnode deleted\n");
   }
   else
   {  ptr = start;
      while(ptr -> next != start)
      {
         ptr = ptr -> next;
      }
      ptr->next = start->next;
      free(start);
      start = ptr->next;
      printf("\nnode deleted\n");
   }
```

3




Deletion in circular singly linked list at beginning

## 5.4 DELETION IN CIRCULAR SINGLY LINKED LIST AT THE END

There are three scenarios of deleting a node in circular singly linked list at the end.

**CASE 1: The list is Empty**

If the list is empty then the condition **start == NULL** will become true, in this case, we just need to print **underflow** on the screen and make exit.

**CASE 2: The list contains single element**

If the list contains single node, then, the condition start → next == start will become true. In this case, we need to delete the entire list and make the start pointer free.

**CASE 3: The list contains more than one element**

If the list contains more than one element, then in order to delete the last element, we need to reach the last node. We also need to keep track of the second last node of the list. For this purpose, the

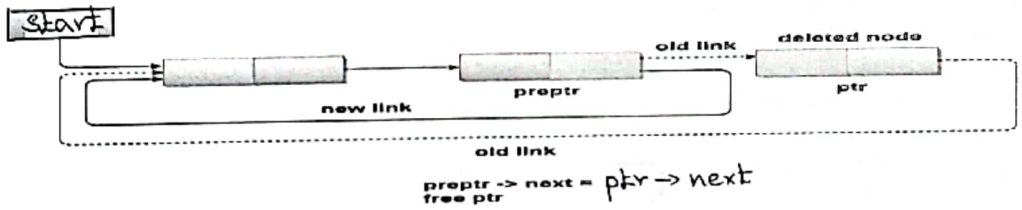


Scanned with OKEN Scanner

two pointers ptr and preptr are defined. Now, we need to make just one more pointer adjustment. We need to make the next pointer of preptr point to the next of ptr (i.e. start) and then make pointer ptr free.

**Pseudocode:**

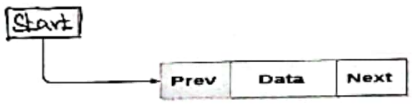
```
void last_delete()
{
   struct node *ptr, *preptr;
   if(start==NULL)
   {
      printf("\nUNDERFLOW");
   }
   else if (start ->next == start)
   {
      start = NULL;
      free(start);
      printf("\nnode deleted\n");
   }
   else
   {
      ptr = start;
      while(ptr ->next != start)
      {
         preptr=ptr;
         ptr = ptr->next;
      }
      preptr->next = ptr -> next;
      free(ptr);
      printf("\nnode deleted\n");
   }
}
```




**Deletion in circular singly linked list at end**

## 6. DOUBLY LINKED LIST

Doubly linked list is a type of linked list in which a node contains a pointer to the previous as well as the next node in the sequence. Therefore, in a doubly linked list, a node consists of three parts: node **data**, pointer to the **next** node, pointer to the **previous** node.




**Node**

**STRUCTURE OF A NODE IN DOUBLY LINKED LIST:**

```
struct node
{
   struct node *prev;
   int data;
```



Scanned with OKEN Scanner

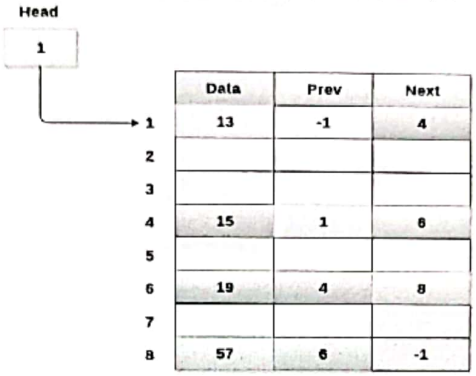
```
    struct node *next;
};
struct node *start=NULL;
```

The **prev** part of the first node and the **next** part of the last node will always contain null indicating end in each direction.




Doubly Linked List

## MEMORY REPRESENTATION OF A DOUBLY LINKED LIST




Memory Representation of a Doubly linked list

| S. No | Operation | Description |
|---|---|---|
| 1 | Insertion at beginning | Adding the node into the linked list at beginning. |
| 2 | Insertion at end | Adding the node into the linked list to the end. |
| 3 | Insertion after specified node | Adding the node into the linked list after the specified node. |
| 4 | Deletion at beginning | Removing the node from beginning of the list |
| 5 | Deletion at the end | Removing the node from end of the list. |
| 6 | Deletion of the node having given data | Removing the node which is present just after the node containing the given data. |
| 7 | Searching | Comparing each node data with the item to be searched and return the location of the item in the list if the item found else return null. |
| 8 | Traversing | Visiting each node of the list at least once in order to perform some specific operation like searching, sorting, display, etc. |

### 6.1 INSERTION IN DOUBLY LINKED LIST AT BEGINNING

There are two scenarios of inserting any element into doubly linked list. Either the list is empty or it contains at least one element.

Allocate the space for the new node in the memory. This will be done by using the following statement.

**newnode = (struct node *)malloc(sizeof(struct node));**

**CASE 1: List is Empty**



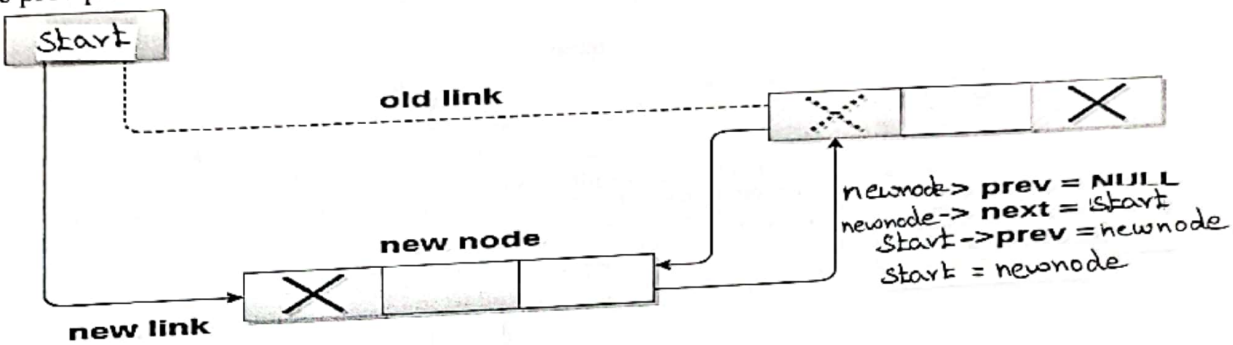
Scanned with OKEN Scanner

Check whether the list is empty or not. The list is empty if the condition **start == NULL** holds. In that case, the node will be inserted as the only node of the list and therefore the prev and the next pointer of the node will point to NULL and the start pointer will point to this node.

**CASE 2: List contains at least one element**

In the second scenario, the condition **start == NULL** becomes false and the node will be inserted in the beginning. The next pointer of the node will point to the existing start pointer of the node. The prev pointer of the existing start will point to the new node being inserted. Since, the node being inserted is the first node of the list and therefore it must contain NULL in its prev pointer. Hence assign null to its previous part and make the start point to this node.

Since, the node being inserted is the first node of the list and therefore it must contain NULL in its prev pointer. Hence assign null to its previous part and make the head point to this node.




**Insertion into doubly linked list at beginning**

**Pseudocode:**

```
void insertion_beginning()
{
  struct node *newnode;
  newnode = (struct node *)malloc(sizeof(struct node));
  printf("\nEnter value for the newnode: ");
  scanf("%d",&newnode->data);
  if(start==NULL)
  {
    newnode->next = NULL;
    newnode->prev=NULL;
    start=newnode;
  }
  else
  {
    newnode->prev=NULL;
    newnode->next = start;
    start->prev=newnode;
    start=newnode;
  }
}
```

**6.2 INSERTION IN DOUBLY LINKED LIST AT THE END**

Allocate the memory for the new node. Make the pointer ptr point to the new node being inserted.

**newnode= (struct node *) malloc(sizeof(struct node));**

**CASE 1: List is empty**

Check whether the list is empty or not. The list is empty if the condition **start ==**



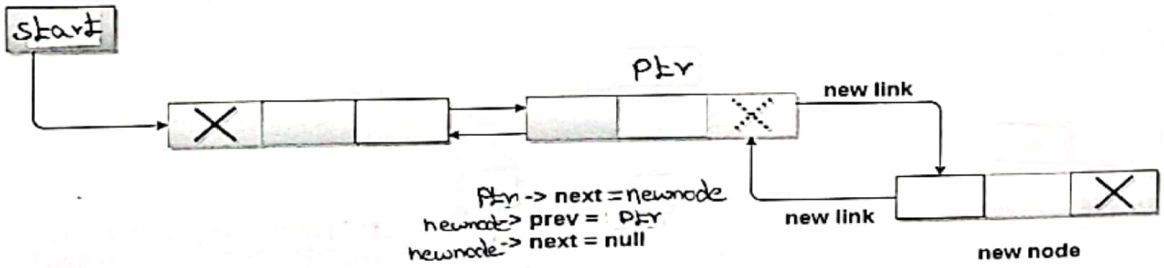
Scanned with OKEN Scanner

NULL holds. In that case, the node will be inserted as the only node of the list and therefore the prev and the next pointer of the node will point to NULL and the start pointer will point to this node.

**CASE 2: List contains at least one element**

In the second scenario, the condition **start == NULL** becomes false. The new node will be inserted as the last node of the list. For this purpose, we have to traverse the whole list in order to reach the last node of the list. Initialize the pointer ptr to head and traverse the list by using this pointer.

The pointer ptr point to the last node at the end of this while loop. Now, we just need to make a few pointer adjustments to insert the new node to the list. First, make the next pointer of ptr point to the new node being inserted. Make the previous pointer of the newnode point to the existing last node of the list. Make the next pointer of the newnode point to the null as it will be the new last node of the list.




**Insertion into doubly linked list at the end**

**Pseudocode:**

```
void insertion_last()
{
  struct node *newnode, *ptr;
  newnode = (struct node *)malloc(sizeof(struct node));
  printf("\nEnter value for the newnode: ");
  scanf("%d",&newnode->data);
  if(start == NULL)
  {
    newnode->next = NULL;
    newnode->prev = NULL;
    start = newnode;
  }
  else
  {
    ptr = start;
    while(ptr->next!=NULL)
    {
      ptr = ptr->next;
    }
    ptr->next = newnode;
    newnode ->prev=ptr;
    newnode->next = NULL;
  }
}
```

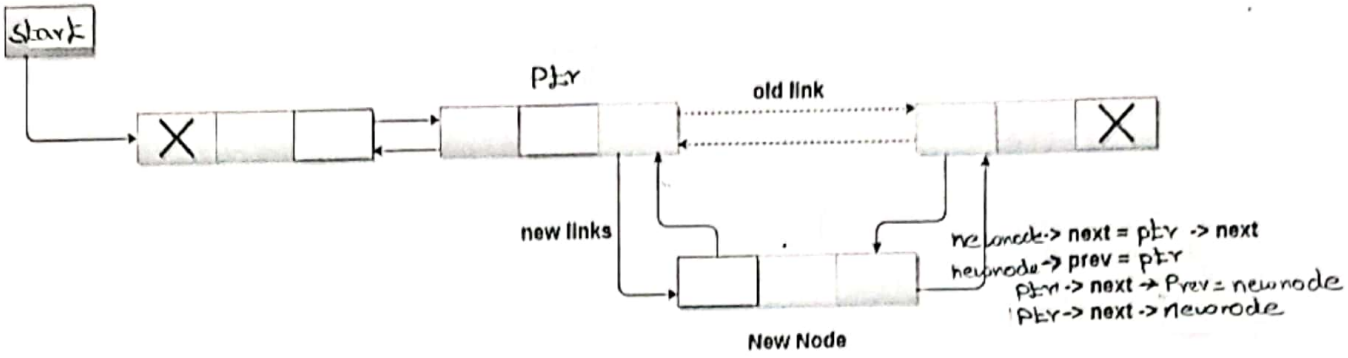
**6.3 INSERTION IN DOUBLY LINKED LIST AT SPECIFIED NODE**

- Traverse the list by using the pointer **ptr** to skip the required number of nodes in order to reach the specified node.



Scanned with OKEN Scanner

- The ptr would point to the specified node at the end of the loop. The new node needs to be inserted after this node therefore we need to make a pointer adjustment here. Make the next pointer of **newnode** point to the next node of ptr. Make the **prev** of the new node point to ptr. Make the **next** pointer of ptr point to the new node. Make the **previous** pointer of the next node of ptr point to the new node.




**Insertion into doubly linked list at the specified node**

**Pseudocode:**

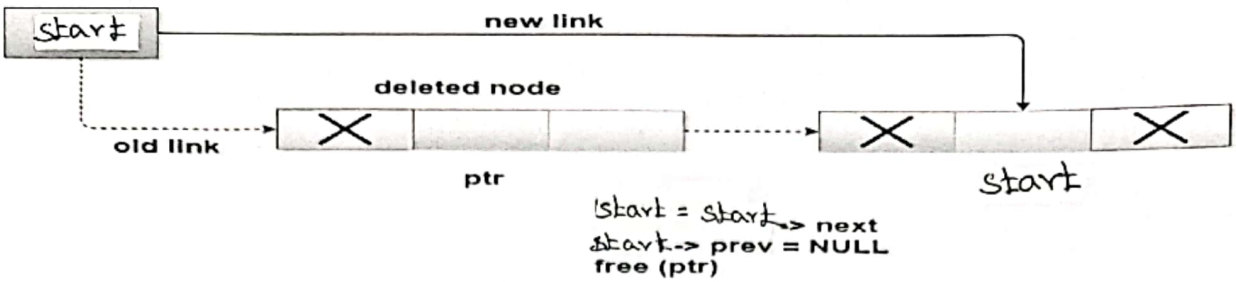
```
void insertion_specified()
{
   struct node *newnode,*ptr;
   int poc,i;
   newnode = (struct node *)malloc(sizeof(struct node));
   printf("\nEnter value for the newnode: ");
   scanf("%d",&newnode->data);
   ptr=start;
   printf("Enter the location ");
   scanf("%d",&poc);
   for(i=0;i<poc-1;i++)
   {
      ptr = ptr->next;
      if(ptr == NULL)
      {
         printf("\n There are less than %d elements", poc);
         return;
      }
   }
   newnode->next = ptr->next;
   newnode -> prev = ptr;
   ptr->next->prev=newnode;
   ptr->next = newnode;
}
```

## 6.4 DELETION AT BEGINNING IN DOUBLY LINKED LIST

- Deletion operation copy the start pointer to pointer ptr and shift the start pointer to its next.
  **Ptr = start;**
  **start = start → next;**
- now make the prev of this new head node point to NULL.
  **start -> prev = NULL;**
- Now free the pointer ptr by using the **free** function.
  **free(ptr);**



Scanned with OKEN Scanner




**Deletion in doubly linked list from beginning**

**Pseudocode:**

```
void deletion_beginning()
{
   struct node *ptr;
   if(start == NULL)
   {
      printf("\n UNDERFLOW");
   }
   else if(start->next == NULL)
   {
      start = NULL;
      free(start);
   }
   else
   {
      ptr = start;
      start = start -> next;
      start -> prev = NULL;
      free(ptr);
   }
}
```

### 6.5 DELETION IN DOUBLY LINKED LIST AT THE END

Deletion of the last node in a doubly linked list needs traversing the list in order to reach the last node of the list and then make pointer adjustments at that position.

In order to delete the last node of the list, we need to follow the following steps.

**CASE 1:** If the list is already empty then the condition **start == NULL** will become true and therefore the operation cannot be carried on.

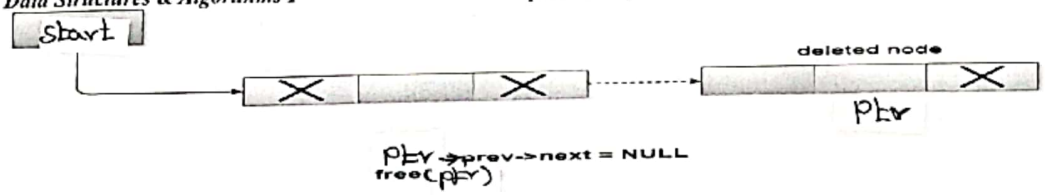
**CASE 2:** If there is only one node in the list then the condition **start → next == NULL** becomes true. In this case, we just need to assign the start of the list to NULL and free start in order to completely delete the list.

**CASE 3:** Otherwise, just traverse the list to reach the last node of the list. This will be done by using the following statements.

- The ptr would point to the last node of the list. Just make the next pointer of the previous node of **ptr** to **NULL**.
- free the pointer as this the node which is to be deleted.



Scanned with OKEN Scanner

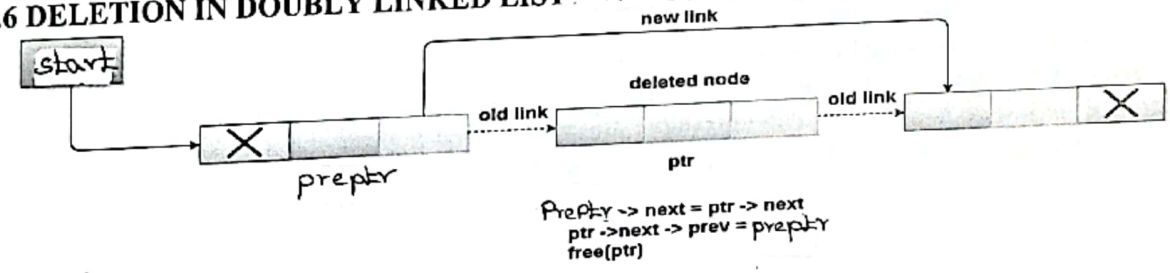



Deletion in doubly linked list at the end

**Pseudocode:**

```
void deletion_last()
{
   struct node *ptr;
   if(start == NULL)
   {
      printf("\n UNDERFLOW");
   }
   else if(start->next == NULL)
   {
      start = NULL;
      free(start);
   }
   else
   {
      ptr = start;
      while(ptr->next != NULL)
      {
         ptr = ptr -> next;
      }
      ptr -> prev -> next = NULL;
      free(ptr);
   }
}
```

## 6.6 DELETION IN DOUBLY LINKED LIST AT THE SPECIFIED NODE




Deletion of a specified node in doubly linked list

**Pseudocode:**

```
void deletion_specified()
{
   struct node *ptr, *preptr;
   int i,pos;
   if(start==NULL)
   {
      printf("\n Underflow");
      exit(0);
   }
```

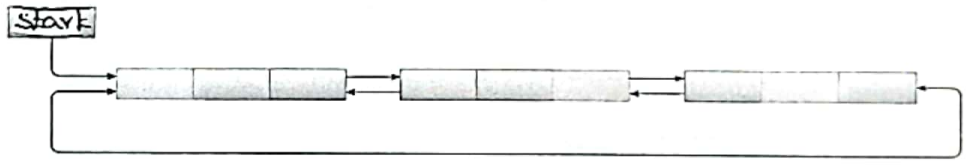


Scanned with OKEN Scanner

```
    else
    {
      printf("\n Enter the position of the node to be deleted : ");
      scanf("%d", &pos);
      if(pos==0)
      {
        ptr = start;
        start=start->next;
        start->prev=NULL;
        free(ptr);
      }
      else
      {
        ptr = start;
        for(i=0;i<pos;i++)
        {
          preptr=ptr;
          ptr=ptr->next;
          if(ptr==NULL)
          {
            printf("\nPosition not Found:\n");
            return;
          }
        }
        preptr->next=ptr->next;
        ptr->next->prev=preptr;
        printf("\nThe deleted element is:%d\t",ptr->data);
        free(ptr);
      }
    }
}
```

## 7. CIRCULAR DOUBLY LINKED LIST

Circular doubly linked list is a more type of data structure in which a node contains pointers to its previous node as well as the next node. Circular doubly linked list doesn't contain NULL in any of the node. The last node of the list contains the address of the first node of the list. The first node of the list also contains address of the last node in its previous pointer.

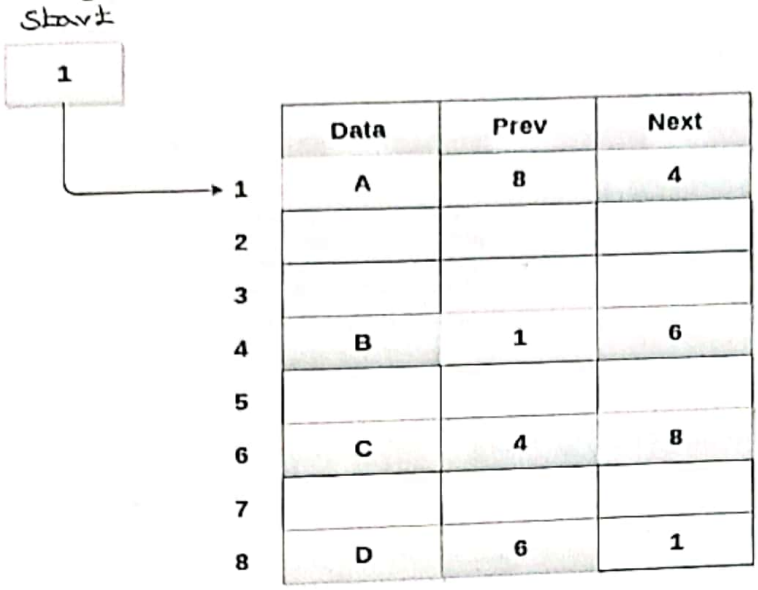



Circular Doubly Linked List



Scanned with OKEN Scanner

## Memory Management of Circular Doubly linked list




Memory Representation of a Circular Doubly linked list

## OPERATIONS ON CIRCULAR DOUBLY LINKED LIST:

| S. No | Operation | Description |
|---|---|---|
| 1 | Insertion at beginning | Adding a node in circular doubly linked list at the beginning. |
| 2 | Insertion at end | Adding a node in circular doubly linked list at the end. |
| 3 | Deletion at beginning | Removing a node in circular doubly linked list from beginning. |
| 4 | Deletion at end | Removing a node in circular doubly linked list at the end. |

### 7.1 INSERTION IN CIRCULAR DOUBLY LINKED LIST AT BEGINNING

Allocate the memory space for the new node by using the following statement.

**newnode = (struct node *)malloc(sizeof(struct node));**

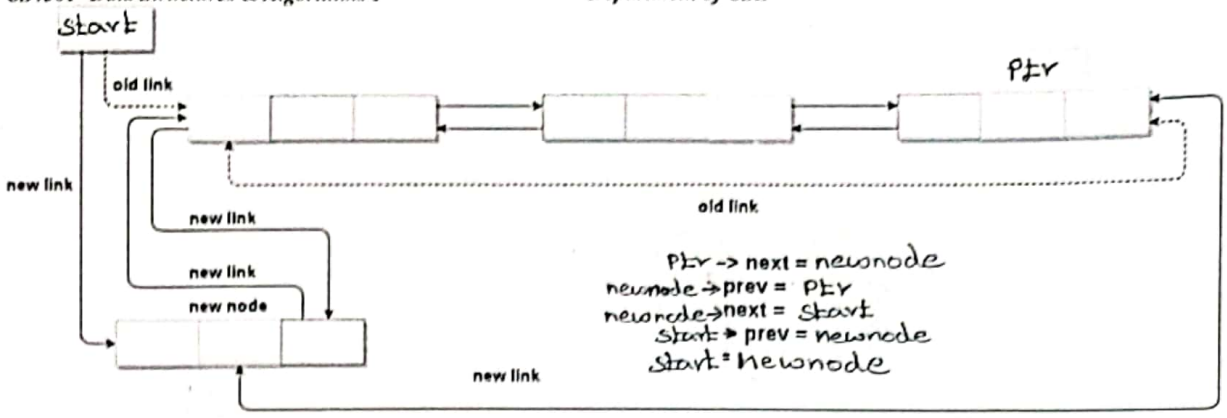
**CASE 1:** In the first case, the condition **start == NULL** becomes true therefore, the node will be added as the first node in the list. The next and the previous pointer of this newly added node will point to itself only.

**CASE 2:** In the second scenario, the condition **start == NULL** becomes false. In this case, we need to make a few pointer adjustments at the end of the list. For this purpose, we need to reach the last node of the list through traversing the list.

At the end of loop, the pointer ptr would point to the last node of the list. Since the node which is to be inserted will be the first node of the list.



Scanned with OKEN Scanner




**Insertion into circular doubly linked list at beginning**

**Pseudocode:**

```
void insertion_beginning()
{
   struct node *newnode,*ptr;
   int item;
   newnode = (struct node *)malloc(sizeof(struct node));
   printf("\nEnter data value: ");
   scanf("%d",&item);
   newnode->data=item;
   if(start==NULL)
   {
      start = newnode;
      newnode -> next = start;
      newnode -> prev = start;
   }
   else
   {
      ptr = start;
      while(ptr -> next != start)
      {
         ptr = ptr -> next;
      }
      ptr -> next = newnode;
      newnode -> prev = ptr;
      start -> prev = newnode;
      newnode -> next = start;
      start = newnode;
   }
}
```

## 7.2 INSERTION IN CIRCULAR DOUBLY LINKED LIST AT END

Allocate the memory space for the new node by using the following statement.

**newnode = (struct node *)malloc(sizeof(struct node));**

**CASE 1:** In the first case, the condition **start == NULL** becomes true therefore, the node will be added as the first node in the list. The next and the previous pointer of this newly added node will point to itself only.

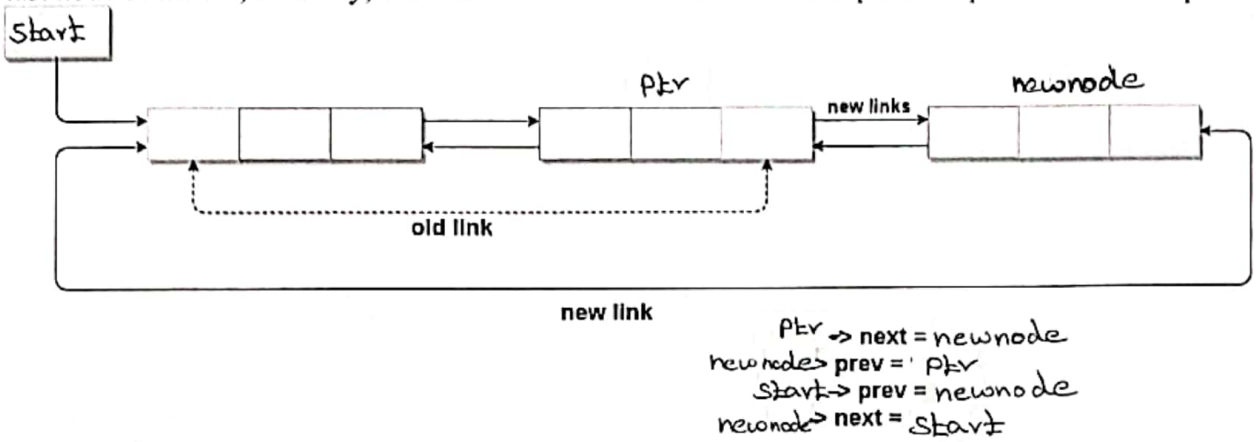
**CASE 2:** In the second scenario, the condition **start == NULL** become false, therefore node will be added as the last node in the list. For this purpose, we need to make a few pointer adjustments in the list



Scanned with OKEN Scanner

at the end. Since, the new node will contain the address of the first node of the list therefore we need to make the next pointer of the last node point to the start node of the list. Similarly, the previous pointer of the start node will also point to the new last node of the list.

Now, we also need to make the next pointer of the existing last node of the list point to the new last node of the list, similarly, the new last node will also contain the previous pointer to the temp.




### Insertion into circular doubly linked list at end

**Pseudocode:**

```
void insertion_last()
{
   struct node *newnode,*ptr;
   int item;
   newnode = (struct node *) malloc(sizeof(struct node));
   printf("\nEnter data value: ");
   scanf("%d",&item);
   newnode->data=item;
   if(start == NULL)
   {
      start = newnode;
      newnode -> next = start;
      newnode -> prev = start;
   }
   else
   {
      ptr = start;
      while(ptr->next !=start)
      {
         ptr = ptr->next;
      }
      ptr->next = newnode;
      newnode ->prev=ptr;
      start -> prev = newnode;
      newnode -> next = start;
   }
}
```

## 7.3 DELETION IN CIRCULAR DOUBLY LINKED LIST AT BEGINNING

**CASE 1:** The node which is to be deleted can be the only node present in the linked list. In this case, the condition start → next == start will become true, therefore the list needs to be completely deleted.



Scanned with OKEN Scanner

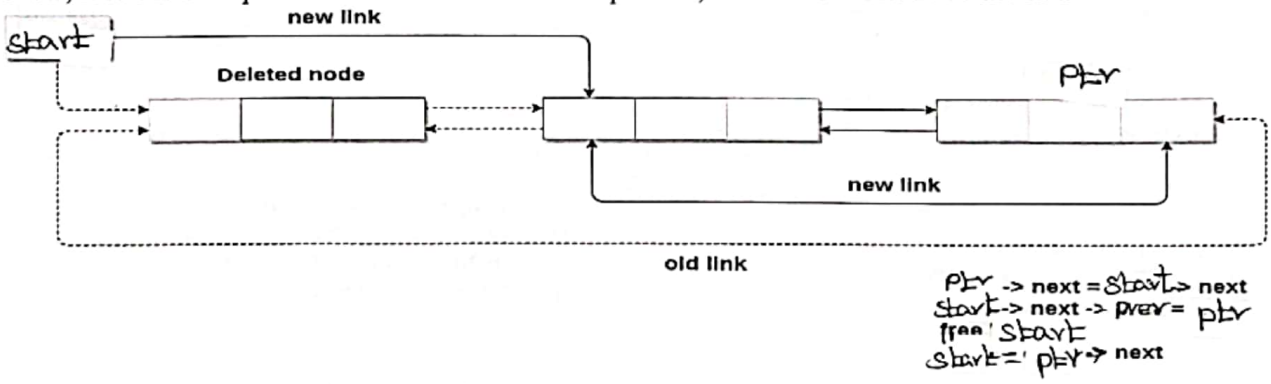
It can be simply done by assigning head pointer of the list to null and free the start pointer.

**CASE 2:** The list contains more than one element in the list, therefore the condition **start → next == start** will become false. Now, reach the last node of the list and make a few pointer adjustments there.

Now, ptr will point to the last node of the list. The first node of the list i.e., pointed by start pointer, will need to be deleted. Therefore, the last node must contain the address of the node that is pointed by the next pointer of the existing start node.

The new start node i.e., next of the existing start node must also point to the last node of the list through its previous pointer.

Now, free the start pointer and the make its next pointer, the new start node of the list.




**Deletion in circular doubly linked list at beginning**

**Pseudocode:**

```
void deletion_beginning()
{
   struct node *ptr;
   if(start == NULL)
   {
      printf("\n UNDERFLOW");
   }
   else if(start->next == start)
   {
      start = NULL;
      free(start);
      printf("\nnode deleted\n");
   }
   else
   {
      ptr = start;
      while(ptr -> next != start)
      {
         ptr = ptr -> next;
      }
      ptr -> next = start -> next;
      start -> next -> prev = ptr;
      free(start);
      start = ptr -> next;
   }
}
```

## 7.4 DELETION IN CIRCULAR DOUBLY LINKED LIST AT END

**CASE 1:** The node which is to be deleted can be the only node present in the linked list. In this case, the condition **start → next == start** will become true, therefore the list needs to be completely deleted.

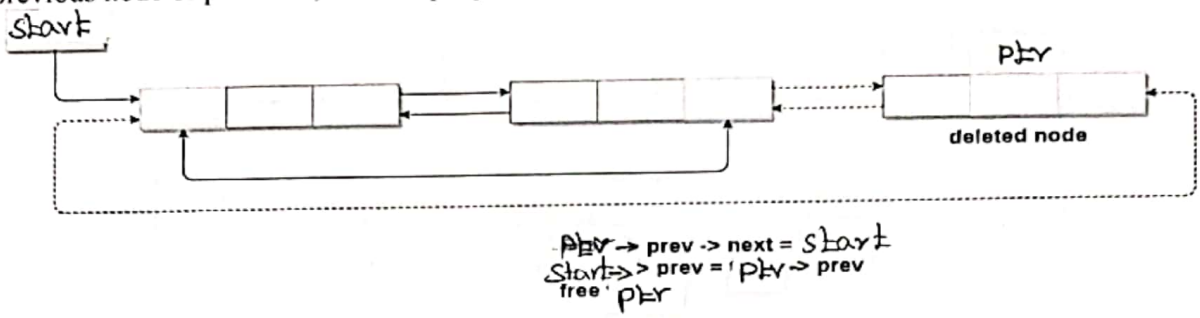
It can be simply done by assigning start pointer of the list to null and free the start pointer.



Scanned with OKEN Scanner

**CASE 2:** The list contains more than one element in the list, therefore the condition **start → next == start** will become false. Now, reach the last node of the list and make a few pointer adjustments there.

Now, ptr will point to the node which is to be deleted from the list. Make the next pointer of previous node of ptr, point to the start node of the list. Make the previous pointer of the start node, point to the previous node of ptr. Now, free the ptr pointer to free the memory taken by the node.




PTR -> prev -> next = start
start -> prev = PTR -> prev
free PTR

**Deletion in circular doubly linked list at beginning**

**Pseudocode:**

```
void deletion_last()
{
   struct node *ptr;
   if(start == NULL)
   {
      printf("\n UNDERFLOW");
   }
   else if(start->next == start)
   {
      start = NULL;
      free(start);
      printf("\nnode deleted\n");
   }
   else
   {
      ptr = start;
      while(ptr->next != start)
      {
         ptr = ptr -> next;
      }
      ptr -> prev -> next = start;
      start -> prev = ptr -> prev;
      free(ptr);
      printf("\nnode deleted\n");
   }
}
```

## 8. APPLICATION OF LINKED LIST - POLYNOMIAL MANIPULATION

Linked lists can be used to represent polynomials and the different operations that can be performed on them.

**Polynomial Representation**

**Consider a polynomial 6x3 + 9x2 + 7x + 1.**

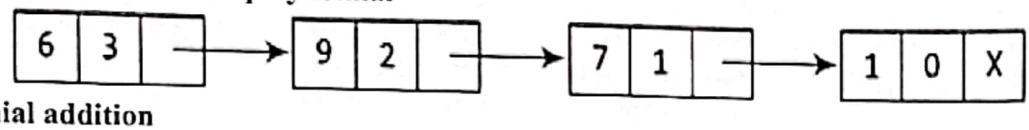
Every individual term in a polynomial consists of two parts, a coefficient and a power. Here, 6, 9, 7, and 1 are the coefficients of the terms that have 3, 2, 1, and 0 as their powers respectively. Every term



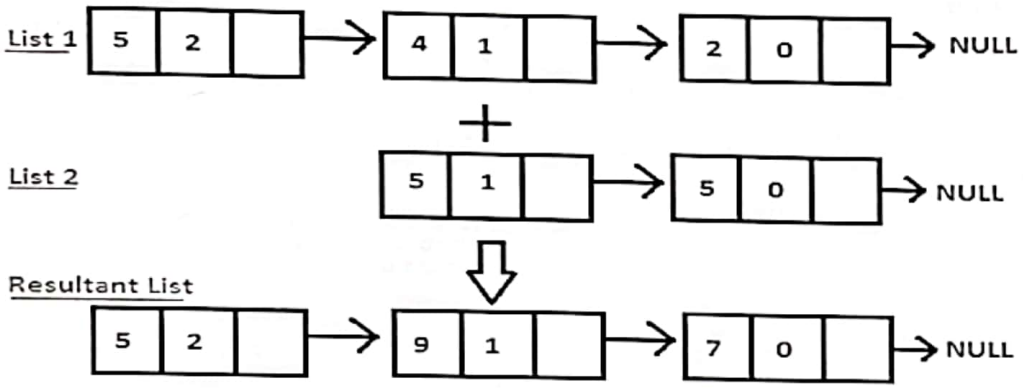
Scanned with OKEN Scanner

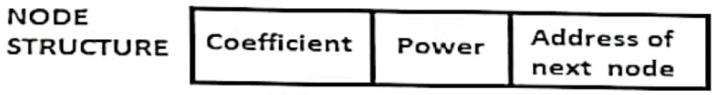
of a polynomial can be represented as a node of the linked list

**Linked representation of a polynomial**




**Polynomial addition**




**PROGRAM:**

```
#include<stdio.h>
#include<stdlib.h>
struct node
{
   int coeff;
   int pow;
   struct node *next;
};
struct node *poly1=NULL,*poly2=NULL,*poly=NULL;
void create(struct node *node)
{
   int n,i;
   printf("Enter number of terms: ");
   scanf("%d",&n);
   for(i=0;i<n;i++)
   {
      printf("Enter coeff: ");
      scanf("%d",&node->coeff);
      printf("Enter power: ");
      scanf("%d",&node->pow);
      node->next=(struct node*)malloc(sizeof(struct node));
      node=node->next;
      node->next=NULL;
   }
}
void show(struct node *node)
{
   while(node->next!=NULL)
   {
      printf("%dx^%d",node->coeff,node->pow);
```



Scanned with OKEN Scanner

```
        node=node->next;
        if (node->next!=NULL)
            printf("+");
    }
}
void polyadd(struct node *poly1,struct node *poly2,struct node *poly)
{
    while(poly1->next!=NULL && poly2->next!=NULL)
    {
        if(poly1->pow>poly2->pow)
        {
            poly->pow=poly1->pow;
            poly->coeff=poly1->coeff;
            poly1=poly1->next;
        }
        else if(poly1->pow<poly2->pow)
        {
            poly->pow=poly2->pow;
            poly->coeff=poly2->coeff;
            poly2=poly2->next;
        }
        else
        {
            poly->pow=poly1->pow;
            poly->coeff=poly1->coeff+poly2->coeff;
            poly1=poly1->next;
            poly2=poly2->next;
        }
        poly->next=(struct node *)malloc(sizeof(struct node));
        poly=poly->next;
        poly->next=NULL;
    }
    while(poly1->next!=NULL || poly2->next!=NULL)
    {
        if(poly1->next!=NULL)
        {
            poly->pow=poly1->pow;
            poly->coeff=poly1->coeff;
            poly1=poly1->next;
        }
        if(poly2->next!=NULL)
        {
            poly->pow=poly2->pow;
            poly->coeff=poly2->coeff;
            poly2=poly2->next;
        }
        poly->next=(struct node *)malloc(sizeof(struct node));
        poly=poly->next;
        poly->next=NULL;
    }
}
void polysub(struct node *poly1,struct node *poly2,struct node *poly)
```



Scanned with OKEN Scanner

```
{
  while(poly1->next != NULL && poly2->next != NULL)
  {
    if(poly1->pow>poly2->pow)
    {
      poly->pow=poly1->pow;
      poly->coeff=poly1->coeff;
      poly1=poly1->next;
    }
    else if(poly1->pow<poly2->pow)
    {
      poly->pow=poly2->pow;
      poly->coeff=poly2->coeff;
      poly2=poly2->next;
    }
    else
    {
      poly->pow=poly1->pow;
      poly->coeff=poly1->coeff-poly2->coeff;
      poly1=poly1->next;
      poly2=poly2->next;
    }
    poly->next=(struct node *)malloc(sizeof(struct node));
    poly=poly->next;
    poly->next=NULL;
  }
  while(poly1->next != NULL || poly2->next != NULL)
  {
    if(poly1->next != NULL)
    {
      poly->pow=poly1->pow;
      poly->coeff=poly1->coeff;
      poly1=poly1->next;
    }
    if(poly2->next != NULL)
    {
      poly->pow=poly2->pow;
      poly->coeff=poly2->coeff;
      poly2=poly2->next;
    }
    poly->next=(struct node *)malloc(sizeof(struct node));
    poly=poly->next;
    poly->next=NULL;
  }
}
void main()
{
  poly1=(struct node *)malloc(sizeof(struct node));
  poly2=(struct node *)malloc(sizeof(struct node));
  poly=(struct node *)malloc(sizeof(struct node));
  printf("\nEnter values for Poly1\n");
  create(poly1);
```



Scanned with OKEN Scanner

```
    printf("\nEnter values for Poly2\n");
    create(poly2);
    printf("\nPoly 1: ");
    show(poly1);
    printf("\nPoly 2: ");
    show(poly2);
    polyadd(poly1,poly2,poly);
    printf("\nAdded polynomial:");
    show(poly);
    polysub(poly1,poly2,poly);
    printf("\nSubtracted polynomial:");
    show(poly);
}
```

**OUTPUT**

```
Enter values for Poly1
Enter number of terms: 2
Enter coeff: 10
Enter power: 2
Enter coeff: 10
Enter power: 1

Enter values for Poly2
Enter number of terms: 3
Enter coeff: 1
Enter power: 3
Enter coeff: 2
Enter power: 2
Enter coeff: 3
Enter power: 0

Poly 1: 10x^2+10x^1
Poly 2: 1x^3+2x^2+3x^0
Added polynomial:1x^3+12x^2+10x^1+3x^0
Subtracted polynomial:1x^3+8x^2+10x^1+3x^0
```



Scanned with OKEN Scanner